Codeindex · 让大模型更好地理解你的代码

原创 跨端技术团队 大淘宝技术 2026年4月20日 02:13 浙江 696人

本文介绍了一款专为解决大模型在处理大型代码仓库时面临的上下文理解难题而设计的工具Codeindex。针对代码量大、分支多及依赖关系复杂等痛点，Codeindex 提供了代码语义化索引、检索以及函数依赖图生成能力。其核心技术亮点包括：利用增量索引与摘要生成技术提升大模型对代码意图的理解，采用分层架构与图数据库（KuzuDB/Postgres）精准构建函数上下游依赖关系。该工具通过 OpenAPI 和 SDK 两种形式，支持 CodeWiz 代码检索、AICR 智能代码审查及 CodeWiki 自动生成文档等应用场景，旨在帮助开发者更高效地构建基于代码的 AI 应用。

01 前言

在日常开发工作中你有没有这样的想法：

1. 想自己实现一个 Agent 助手来回答关于代码的一些问题，但是代码仓库过大，都塞给大模型也不太现实...
2. 想对仓库代码做分析，但还要考虑不同仓库不同分支的文件变化...
3. 想识别函数的上下游依赖，使大模型更准确地理解代码，还要单独实现函数依赖的上下游能力...



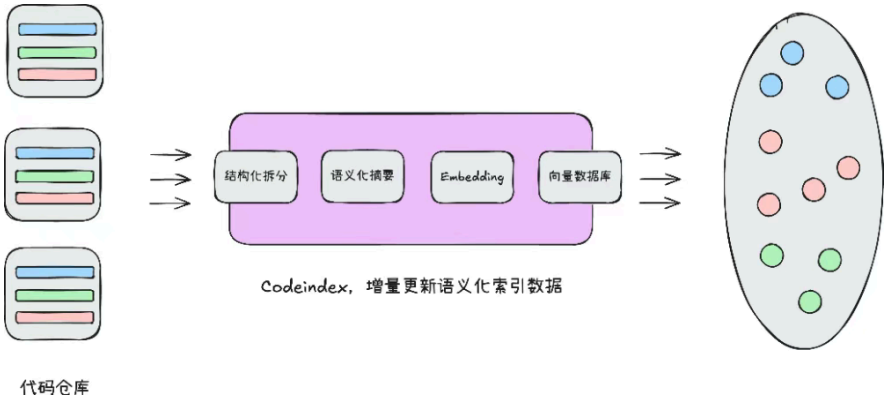
现在，你可以使用 Codeindex 来帮助你解决这些问题：

1. 增量语义化索引&检索你的代码仓库，召回相关的代码片段
2. 增量生成代码片段&文件级别的代码摘要
3. 根据函数声明&调用生成函数依赖关系图，可获取函数依赖的上下游函数

下面就来介绍下 Codeindex 支持的能力以及可以实现的场景。

02 介绍

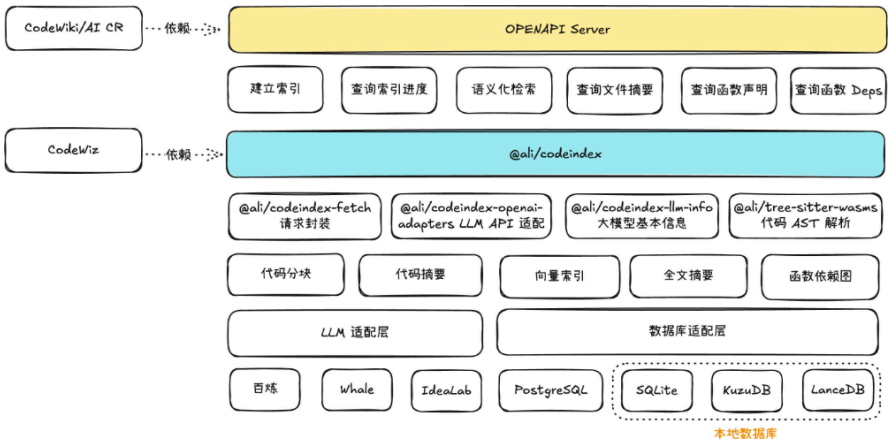
Codeindex 是一个代码语义化索引、检索和函数依赖图生成工具，并支持增量索引。借助 codeindex，你可以对你的大型代码仓库进行索引，通过语义化描述检索代码仓库中的相关代码片段，此外你还可以获取这段代码或者其对应文件的语义化摘要，从而帮助大模型更好地理解代码意图。



03 整体架构

Codeindex 提供 SDK 与 OpenAPI 两种服务，OpenAPI 依赖 Node.js SDK 实现，二者均提供如下能力：

1. 建立索引：根据仓库及分支对代码建立语义化索引，并存储文件哈希值信息，二次索引可复用，实现增量索引。
2. 查询索引进度：因索引过程中涉及文本模型及向量模型的调用，大型项目的索引时间较长，可通过该能力实时查询索引进度。
3. 语义化检索：索引完成之后可通过一段语义化描述检索代码仓库中相关的代码片段及其语义化摘要。
4. 查询文件摘要：可查询单个文件的语义化摘要，可用于 Codewiki 等应用。
5. 查询函数声明：可查询文件中声明的函数片段。
6. 查询函数 Deps：查询某个函数的上下游函数依赖，可用于 AI CR 等场景，辅助大模型判断代码变更是否合理。



04 部分细节解析

语义化摘要

语义化摘要依赖代码 Chunk 拆分的结果，在分块过程中会使用如下策略：

1. codeChunker：`codeChunker` 利用 Tree-sitter 解析器生成 AST，并基于语法结构进行分块：
   a. 若 Class 内代码超出 chunk token 上限，会对内部的函数体省略，保持 Class 的整体代码结构完整
   b. 对 Class 内部的函数成员二次处理，单独进行 Chunk 拆分。
2. basicChunker：`basicChunker` 是一个简单的基于行的分块器，适用于纯文本和 Markdown 文件，会按照 Chunk token 上限进行拆分。

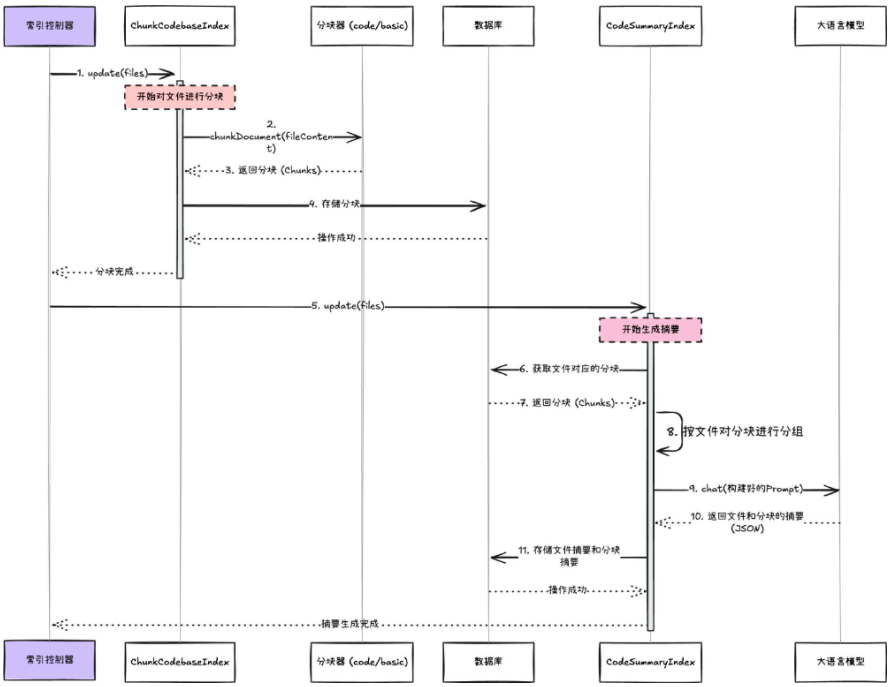
Chunk 拆分完成之后会生成语义化代码摘要，代码片段会按照如下结构发送给大模型，最外层为 document 标签，表示单个文件，path 属性为文件路径；内部的 code 标签为当前文件内拆分的代码片段，`start_line` 与`end_line` 分别表示代码片段的开始行号和结束行号。

```
<document path="lib/utils.js">
  <code start_line="0" end_line="2">
    function formatDate(date) {
      return date.toISOString().split('T')[0];
    }
  </code>
  <code start_line="3" end_line="5">
    function validateEmail(email) {
      return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
    }
```

```
  </code>
</document>
```

我们会将上述信息发送给大模型让其做摘要，返回结果为如下结构的 JSON 对象

```
[
  {
    "summary": "文件维度的代码摘要",
    "path": "文件路径",
    "chunks": [
      {
        "start_line": "开始行号",
        "end_line": "结束行号",
        "summary": "代码片段级别的代码摘要"
      }
    ]
  }
]
```

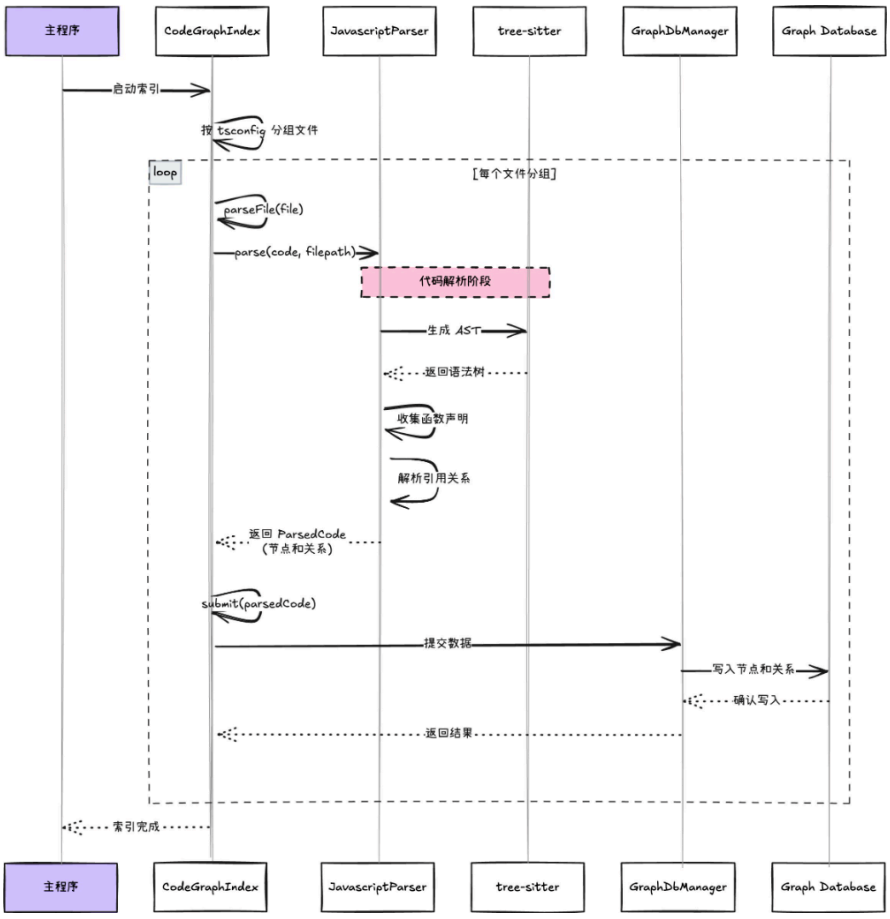


函数依赖图

代码依赖图可以用在 AI CR 等场景中，它可以查询函数依赖的上下游，从而为大模型提供更完备的上下文信息。

由于涉及到多种编程语言以及数据库语法差异，函数依赖图部分整体采用分层架构设计：

1. Parser 适配层，所有语言的依赖图解析均通过拓展该层的方式实现，对外暴露 API 一致，内部采用 tree-sitter 对代码进行解析，分析其内部的函数依赖。
2. GraphDB 适配层，由于涉及到 SDK 与 openapi，故需要考虑本地与线上两种存储介质，所以对 KuzuDB 与 Postgres 数据库对外暴露接口标准化，无缝切换存储介质。



函数依赖图生成时序图

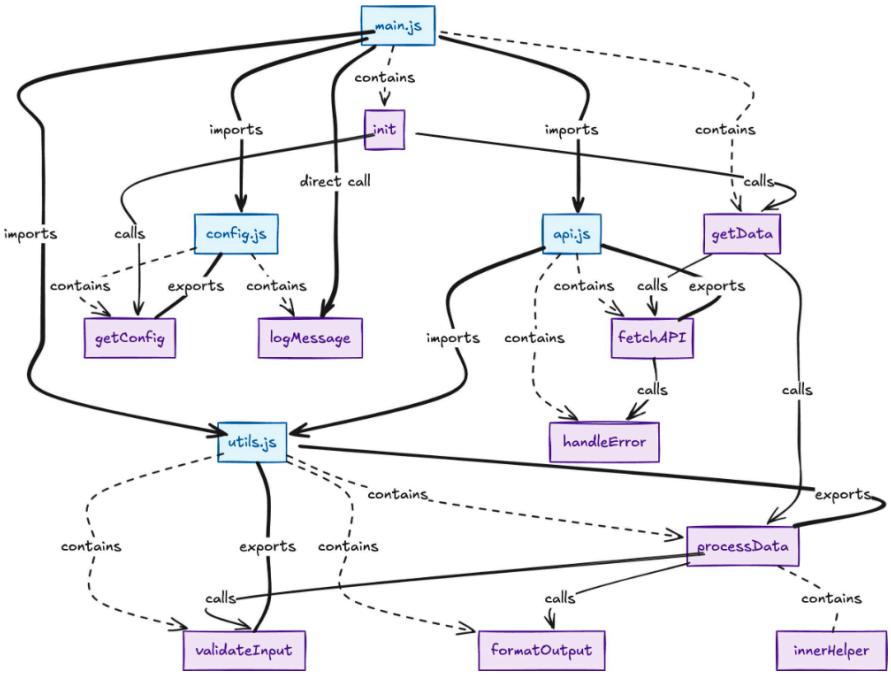
> 总结一下上面的时序图，SDK 内部会查询文件内部声明了哪些函数、函数内部有没有嵌套声明函数、函数内部调用了哪些函数、被调用的函数是来自内部声明还是外部引用。获取到这些信息之后可以合并为图数据结构存储在图数据库中。

图数据存储方面，嵌入式图数据库采用 KuzuDB，线上使用 Postgres Age 插件，二者均设计了如下表结构用于存储图数据结构：

1. Files:节点表，存储代码库中每个文件的基本信息。
2. Functions: 节点表，存储代码中定义的函数信息。
3. Contains: 关系表，表示文件与函数之间的包含关系 (Files -> Functions)。
4. FunctionCalls: 关系表，记录函数之间的调用关系 (Functions -> Functions)。
5. FileCalls: 关系表，记录文件直接调用函数的关系 (Files -> Functions)。
6. Imports: 关系表，表示文件之间的导入/导出关系 (Files -> Files)。
7. Exports: 关系表，表示文件导出函数的关系 (Files -> Functions)。

8. FunctionContains: 关系表，表示函数内部定义了其他函数的关系（嵌套函数），(Functions -> Functions)。

借助上述的图数据表可以查询函数的多级依赖。可以生成类似下面的关系图



05 应用

CodeWiz 集成

CodeWiz 基于 codeindex 的索引/检索能力实现了代码库索引功能，并为大模型提供检索工具，可在 Chat 过程中获取上下文，从而更好地为用户解决问题。

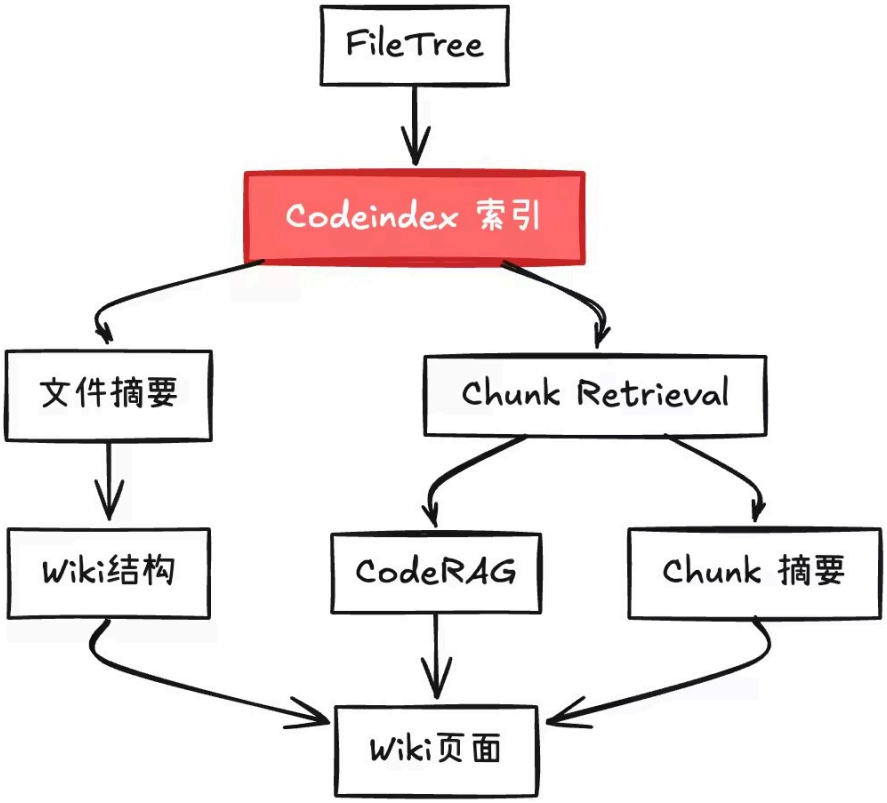
AI CR Agent

AI CR 是聚焦中后台场景、具有智能上下文分析能力的 Agent 应用，其内部依赖了 Codeindex 来获取函数依赖的上下游信息，以帮助大模型更好地判断当前变更是否对已有功能产生影响。

CodeWiki

CodeWiki 可根据仓库中的代码基于 Qwen 大模型 生成 wiki 文档。

Codeindex 索引过程中会生成代码片段以及全文语义化摘要，将文件路径及其摘要发送给大模型可以帮助大模型更准确地输出 Wiki 结构，Codewiki 前置会先生成对应仓库的 Codeindex 索引，后续会基于其拆分的代码片段以及语义化摘要生成文档。



06 总结

总结一下，Codeindex 是一个代码语义化索引&检索工具，并且可以生成代码片段/文件级别的语义化摘要和函数依赖图，你可以使用这些能力来构建你的 AI 应用。目前 Codeindex 同时提供了 OpenAPI 和 SDK 两种方式，OpenAPI 中已经配置好了 SDK，只需要传入你的仓库信息就可以为你的仓库生成索引。

07 团队介绍

本文作者崇野，来自淘天集团-跨端技术团队。本团队服务于淘宝基础用户产品，是淘宝重要的业务线之一。团队以前端、Weex、Native端的技术解决方案框架和研发模式不断完善自己，持续探索端智能等创新，打造极致的体验和工程技术，保障多端设备的适配和稳定运行，致力于让亿级规模的交付能够更丝滑、更稳定。

¤ 拓展阅读 ¤

大淘宝技术
大淘宝技术官方账号
967篇原创内容
公众号

阅读 8429

留言

写留言

还没有留言，来写首评